BuggleBuggle

Allegro 5 · C Language · 2D Platformer

개발 기간: 2026. 03. 25 ~ 2026.03.31

개발 환경

- Language: C
- Library: Allegor5
- IDE: Visual Studio
- 형상 관리: GIT
- 작업 공유: Notion

개발 인원: 4명

- 김진수 (PM / Core)
- 윤성민 (Render / Map)
- 인정원 (Physics / Collision / Player)
- 최이레 (Score / Enemy)

게임 목표

1 **최종 목표** 버블을 발사하여 모든 적을 가두고 처치함으로써 스테이지를 클리어하고 높은 점수를 기록하는 것

2 **승리 조건** 총 3개의 스테이지에 존재하는 모든 적을 섬멸

3 **패배 조건** 적 또는 적의 투사체에 닿아 보유한 3개의 목숨(Heart)을 모두 잃었을 때

핵심 플레이 방식

버블 생명주기

발사: 플레이어의 방향으로 직선 발사
부유: 일정 거리 이동 후 또는
벽 충돌 시 천천히 상승
터짐: 일정 시간 경과 후 소멸

포획 및 처치

발사된 버블에 적이 닿으면 갇힌 상태로 전환
갇힌 적을 플레이어가 몸으로 터뜨리면
최종적으로 처치 완료

점프 및 이동

중력과 점프의 조화를 통한
자연스러운 포물선 구현

1. 게임 소개(3)

주요 오브젝트

POINT01

플레이어

조작: 방향키(이동/점프),
스페이스바(버블 발사)

POINT02

지상 추적형

플레이어의 위치를 파악해 추적 및 확률적 점프 수행

POINT03

공중 투사체형

중력을 무시하며 플레이어 방향으로 투사체를 조준 발사
발사체는 중력을 무시하며 플레이어 방향으로 발사

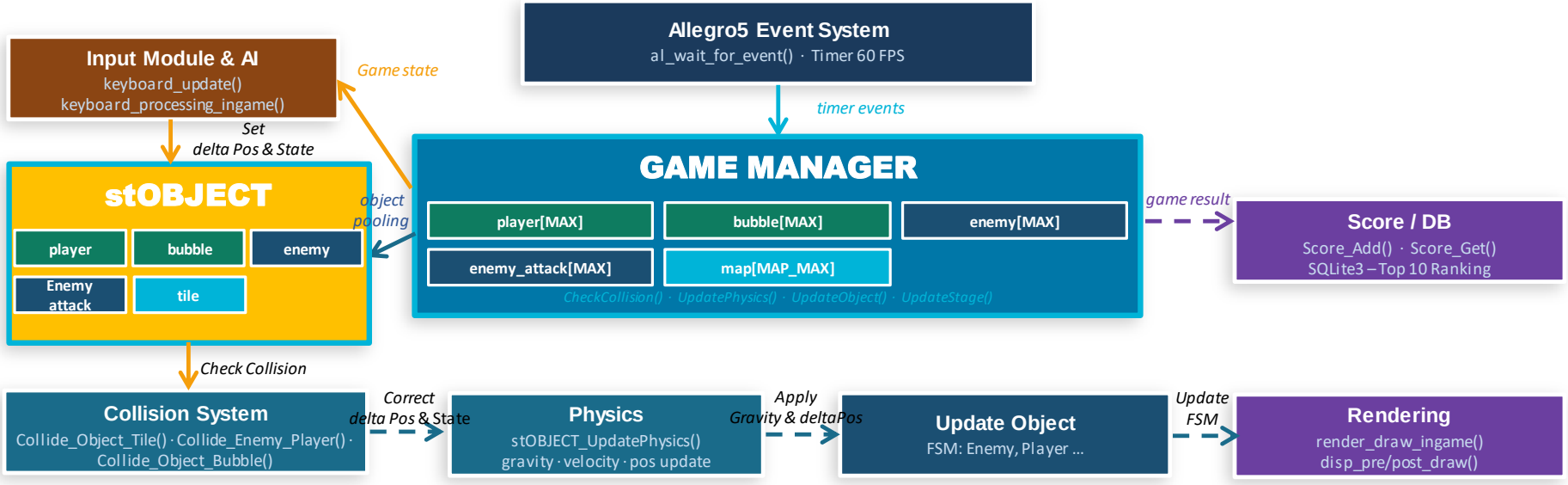
Others

POINT04

환경 및 기타

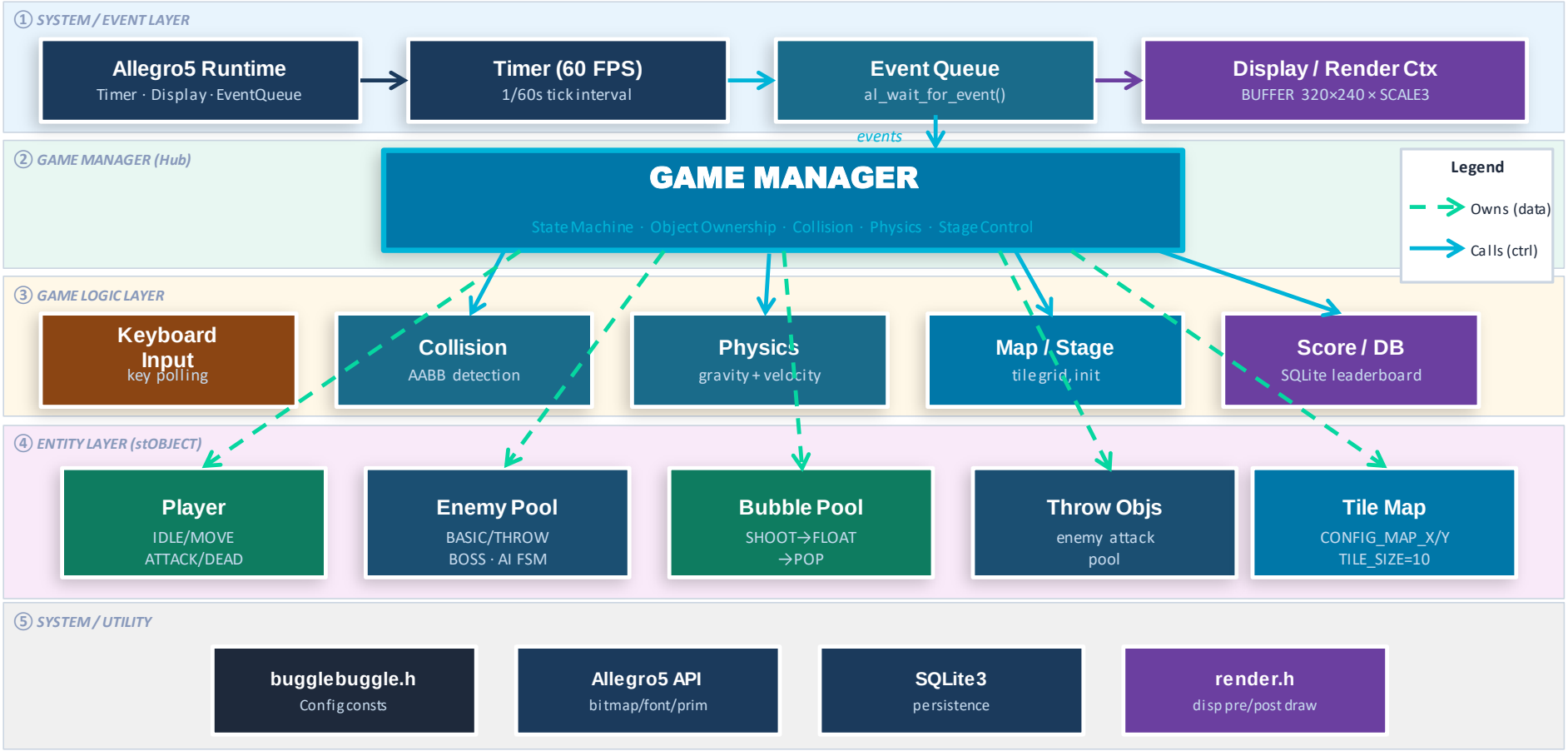
타일: 충돌 판정이 있는 지형지물
투사체: 플레이어 위치를 향해 직선 이동
리더보드: DB를 연동하여 상위 10위까지의 점수와 이름을 관리

2. 핵심 시스템

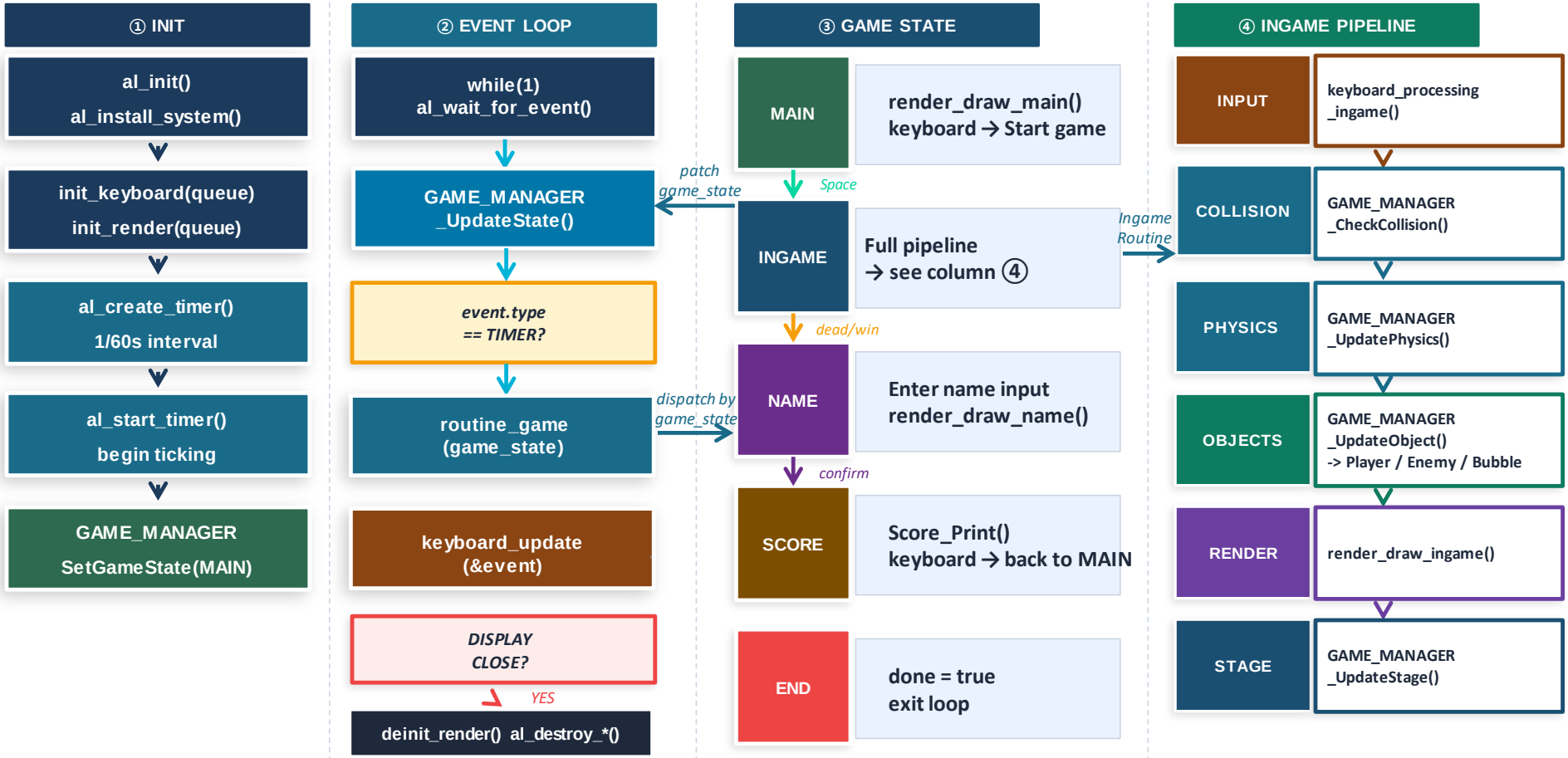


Input & AI	Converts Allegro key events → player speed/direction via player_update_input() & Determine enemy FSM
Collision	AABB all object pairs: Tile↔Objects, Enemy↔Player, Bubble↔Enemy each frame
Physics	Applies gravity accumulation and pos += speed each frame; resets speed.x after
Update Object	FSM: IDLE→MOVE→ATTACK/TRAPPED→DEAD; ground AI jumps, fly AI zigzags
Rendering	Rendering Objects using FSM

3. 시스템 아키텍처


① SYSTEM / EVENT LAYER
Allegro5 Runtime
Timer · Display · EventQueue
Timer (60 FPS)
1/60s tick interval
Event Queue
al_wait_for_event()
Display / Render Ctx
BUFFER 320×240 × SCALE3
events
② GAME MANAGER (Hub)
GAME MANAGER
State Machine · Object Ownership · Collision · Physics · Stage Control
Legend
Owns (data)
Calls (ctrl)
③ GAME LOGIC LAYER
Keyboard Input
key polling
Collision
AABB detection
Physics
gravity + velocity
Map / Stage
tile grid, init
Score / DB
SQLite leaderboard
④ ENTITY LAYER (stOBJECT)
Player
IDLE/MOVE
ATTACK/DEAD
Enemy Pool
BASIC/THROW
BOSS · AI FSM
Bubble Pool
SHOOT→FLOAT
→POP
Throw Objs
enemy attack
pool
Tile Map
CONFIG_MAP_X/Y
TILE_SIZE=10
⑤ SYSTEM / UTILITY
bugglebuggle.h
Config consts
Allegro5 API
bitmap/font/prim
SQLite3
persistence
render.h
disp pre/post draw

4. 실행 흐름


① INIT
al_init()
al_install_system()
init_keyboard(queue)
init_render(queue)
al_create_timer()
1/60s interval
al_start_timer()
begin ticking
GAME_MANAGER
SetGameState(MAIN)
② EVENT LOOP
while(1)
al_wait_for_event()
GAME_MANAGER
_UpdateState()
event.type
== TIMER?
routine_game
(game_state)
keyboard_update
(&event)
DISPLAY
CLOSE?
YES
deinit_render() al_destroy_*()
patch
game_state
dispatch by
game_state
③ GAME STATE
MAIN
render_draw_main()
keyboard → Start game
Space
INGAME
Full pipeline
→ see column ④
dead/win
NAME
Enter name input
render_draw_name()
confirm
SCORE
Score_Print()
keyboard → back to MAIN
END
done = true
exit loop
Ingame
Routine
④ INGAME PIPELINE
INPUT
keyboard_processing
_ingame()
COLLISION
GAME_MANAGER
_CheckCollision()
PHYSICS
GAME_MANAGER
_UpdatePhysics()
OBJECTS
GAME_MANAGER
_UpdateObject()
-> Player / Enemy / Bubble
RENDER
render_draw_ingame()
STAGE
GAME_MANAGER
_UpdateStage()

5. 결과 및 시연